

# Lec 13 Regularization Techniques in AE: Denoising

## Lec 13 Regularization Techniques in AE: Denoising

---

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| <b>Lecture</b>  | 13 / 63 + intro                                        |
| <b>Duration</b> | 42:36                                                  |
| <b>Captions</b> | English (auto-generated)                               |
| <b>Words</b>    | 6,558                                                  |
| <b>Video</b>    | <a href="#">q3XAdXdqz8s</a>                            |
| <b>Playlist</b> | <a href="#">Fundamentals of Generative AI and LLMs</a> |

---

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

### Transcript

**[00:00]** Hello everyone. Welcome to the lecture series on fundamentals of generative AI and the large language models. In our previous session, we had a scene what are the different types of autoencoder. So, in this a particular session today, we will see most important regularization techniques. There are totally three regularization which we'll be seeing under this. One is a denoising, second is a sparse, and third one is contractive. In this particular session, because these are very important topics, I'll be going very slow. So, this particular session will be focusing only on a denoising autoencoder.

**[00:40]** Now, why do we require this particular concept? So far, what in the previous types, we have seen suppose if you are if you are aim is to reconstruct the you know, tabulated data. We can go either with a shallow autoencoder or we can go with a deep autoencoder. Suppose if you are aim is to reconstruct the image, then we can go for CNN based one. So, from our previous session, we have understood that with autoencoder, we can able to reconstruct the data. If that is the case, why do we require this topic all together?

**[01:16]** So, what's your thought process? From that word regularization, what is that we are understanding? Regularization is always restricting the freedom of weight,

you know, those parameters, whatever the parameters that we have for a neural network, that is weights and bias. The way we are updating, the way we are taking care, the movement of those weights, all that is what we are going to restrict it. Now, why to restrict? Why do we require this regularization one? So, let's understand that.

**[01:45]** So, so far, this is what we have seen from our previous one. So, we have an input, that input gets multiplied with the  $a$  and we are adding the bias  $a$  term, and this entire you're passing it to an activation function. And that activation function here we are denoting it as a  $G$ , and that  $G$  because it is at an encoder side, it will be a non-linear activation function. What we are seeing now is a summary once again before we start the next one.

**[02:17]** » [snorts] » That is denoising. So, at the decoder side, what is that we are doing? Whatever is the representation that we have obtained, this  $H$ .  $H$  is getting multiplied with the weight, and then we are adding its associated bias  $a$  term, and this entire you're passing it to an activation function. And here I'm I'm only talking about the last layer. So, at the last layer, the activation function what you'll be using is a decoder activation. And that decoder activation will be at the output layer. That depends on what kind of data that we are reconstructing. For all the hidden layers, you'll be using an ReLU activation that we have seen it already.

**[03:01]** And what is our ultimate aim during the training process? It has to minimize the loss between the  $X$  and  $X$  cap or  $X$  hat. So, this is our original input and the reconstructed one. So, this is all fine when you have a very good autoencoder. Now, let us see what if you have an autoencoder where the architectural diagram is like this. So, what kind of architectural diagram it has? From our previous session, can you able to recall? Is it not an over complete? Why it is called over complete?

**[03:36]** Here, the dimensions of an latent representation is much larger than the input. So, what exactly is the problem? We have already seen there can be an identical mapping. There is no trivial learning here there is no learning here no robust learning. From here you can directly copy the values. So with this where is the learning? So when whenever the hidden layer dimension is a large, the model may simply copy the input to the hidden and from the hidden to an output without any learning meaningful features. Yes, we have seen this. Now in such a case, there is an identical identity mapping instead of useful structure. So this will definitely lead to an overfitting, very poor generalization, and it leads to an

**[04:28]** overfitting a problem. So whenever there is an overfitting, you require a regularization term. So therefore we require and a regularized variant of an autoencoder that is mainly needed. Why? To prevent this identity mapping. And you have to encourage even in a such particular architecture, you should still encourage the meaningful feature learning. So I hope you understood the context why we require an regularization term.

**[04:58]** I'm just giving one scenario where you know the overfitting. The scenario here is whenever you consider an overcomplete architecture, there is a very high possibility of over fitting. That doesn't mean that in undercomplete we will not face. This is a classic scenario where definitely you will have an overfitting problem. So now to avoid this, how is that we will be solving it? So we require an denoising autoencoder.

**[05:27]** So now with this, can you tell what is denoising autoencoder? It is an a regularized variant of an autoencoder, mainly to prevent that overfitting problem or making the model to learn the most important features. Now, [snorts] if you see how exactly denoising autoencoder works. This is what is the architecture. There is an input. Whatever the input that we have, take that input and then corrupt this input. So, the corrupted input is a passed as an input to the encoder. So, what we are doing in a denoising, whatever the input that we have, intentionally we are corrupting it. How are we corrupting it?

**[06:12]** We will see that shortly. So, that corrupted input is a passed as an input to an encoder. And the rest everything will remain the same. The corrupted input when it is passed to an encoder, encoder will start extracting the features. We get the latent representation. From the latent representation, you are passing it to a decoder. But, what is the trick here? When ultimately you have to construct you have to consider the loss. Now, when you calculate the loss, the loss is between the reconstructed one and will it be with the original input or with the corrupted input? What is the What is your take on this? I'm telling, whatever the input that we have, the input is not as it is a passed to an

**[07:01]** model. The input is a first corrupted. So, the corruption can be adding some noise, increasing the brightness, decreasing the brightness. You are somehow, you know, tapping the values. You are changing the values. Somehow you have corrupted the original input. So, that corrupted input, now let us consider it as a tilde  $x$  tilde. This is passed as an input to an encoder model or autoencoder. Towards the end, we are getting an reconstructed value. Let us assume reconstructed one is  $x$ . Since there is a hat and tilde because of the confusion, let's take reconstructed one is  $x$  dash.

**[07:42]** Now, the error should be with respect to this or the error should be compared to the original one. It has to be with the original  $X$ . So, when you compare it with the original  $X$ , definitely input passed is a corrupted one. And you are comparing the ground truth what you're considering is the original clean input. So, with this, is it not the model is a forced to learn? It cannot relax. Are you getting this concept? So, let us understand what I've explained you is the overall working mechanism. Let us go in detail step-by-step how auto denoising autoencoder works. So far what we have understood, » [snorts] » you're taking an input, you're first corrupting the input randomly using what? Using a

**[08:31]** probability-based rule. And then that corrupted input is passed as an input to an autoencoder. And you are expecting the model to learn the features and reconstruct the original clean input, right? So, diagrammatically if you have to see, this is the original input to four, clean input. To that, you're adding the noise and you're getting this noisy image. This noisy image is a passed as an input to an denoising autoencoder. And towards the end, what is your aim? Aim is to get the original clean image. Why original? Because the original image is clean one. For this, you have corrupted it. So, ultimately aim is to get back to that.

**[09:17]** So, when you when your aim is this, when you have an architecture of this nature where hidden layer is larger than an input, can now the model, you know, think of relaxing? It cannot, right? What do you mean by relaxing? Copying the same here and from here back to this. That is called a relaxing. Are you now relaxing here? Can the model relax? No. Why it cannot? Because input is corrupted one and its requirement is to construct the clean. So, it cannot relax. It has to work more carefully.

**[09:55]** That more carefully is what is the regularization. I hope the concept is clear now. Yes. So, with this what we have understood, you have to disturb. You have to randomly corrupt the input. How are we corrupting the input? What are the different techniques? The original clean input is corrupted and you are getting a noisy input. The first way of corruption is corrupting the input is Gaussian noise. Take the input to this input take the Gaussian add the Gaussian noise. The noise is a taken of from the Gaussian distribution whose mean is zero and then the variance. This is the variance. So, now let us assume these are the values that we have obtained. Since there are four input features. Now, can you

**[10:45]** imagine there are four input features which is 0.50, 0.80, 0.20 and 0.90. To this when you add the noise, is it not you have changed the original input now to 0.51, 0.75, 0.23, 0.82. Are they both the same? Definitely no. This is 0.5, this is 51. This is 8, this is 75. This is a 2, this is 23. This is a 9 and this is 82. So, definitely you have corrupted the input. Now, this corrupted input is a passed as an input to an you know encoder and decoder. And ultimately you are expecting to get the output and that output should not match to this, should match it to this one.

**[11:34]** So, when it is when it has to match to the original, you cannot relax with respect to an architecture. The model should be more careful when it is doing this. So, the very first way of corrupting the input is Gaussian. Where now I'm just giving one example where how a small noise has been added to an input. So, not necessarily that you will have such a small disturbance. Yes, you can you will see the value in and around.

**[12:04]** Second way is you know, masking noise. What is a masking noise? Now, let us assume you are taking an input. Here the input what we have considered is this.

Randomly you will set few features to zero. Now, can you do it randomly or do you do it systematically? Suppose if you take the value as 0.4. So, the probability with which you take an feature and then make it as zero, so that value you are fixing. So, it is Q value here is 0.4. So, which means each of this feature there is a 40% chance that it can become zero and 60% there is a chance that it will be unchanged. That is what with the probability Q that particular sample can become zero. With the probability 1 by 1 minus Q, the value will be retained as it is.

**[13:08]** Right? So, it will be the same value with the probability 1 minus Q. So, if you have to understand this, let us assume that Q value be 0.4. So, which means 40% there is a chance that the particular feature becomes zero. 60% there is a chance that the value will retain as it is. Okay? Now, with example if you have to understand, look at this. These are the features that we have. Since Q value is a 40, so [snorts] 40% of five is two.

**[13:42]** Right? So, which means two values in this will become zero. And the rest values will retain as it is. Right? Now, in this, what is the 40% there is a chance that 0.8 will become zero. And 60% there is a chance that it is retained as it is. So, this is retained as it is. This is turned to zero. This is retained. This is zero. This is retained. Now, you can ask on what basis you have chosen only 0.3 and 0.5 to zero. No, it can be one one more sample can be zero, zero, 0.9, 0.5, 0.2.

**[14:24]** One more possibility would be 0.8, zero, zero, 0.5, 0.2. The I'm not saying that take only a specific value. Every feature, there is only 40% chance that it can become zero. 60% it is retained. So, this is one example where second and fourth one turned into zero and the remaining values are as it is. So, that is the second way of you know, um corrupting the input. What is the third one? The most popular, it is a salt and a pepper noise. So, what is a salt and a pepper noise? So, salt is white, pepper is a black. So, you will take the features and you will change that to an extreme intensity.

**[15:13]** What is extreme intensity? Either zero or one. That's what is white and a black, right? So, in a salt and a pepper noise, ultimately our aim is to corrupt the input. One way is add the Gaussian noise. Second is randomly set it to zero. Third is salt and pepper. And in salt and pepper, what are we doing? We will take randomly some values and we will change that with extreme intense values, not with a zero or zero. We are changing it to extreme values. Let's take an example.

**[15:46]** Suppose if you consider in a binary image. So, here I'm considering an image with a very small 3 cross 3. Now, here you see one is being changed to zero and a zero is being changed to one. We are not flipping. Please don't get it wrong. Here we are changing the pixel values to its extreme intensity values. What is extreme intensity? For one, it is zero. For zero, it is one. So, that's how you are going to, you know, some pixels have been flipped here where one is changed to zero and zero is changed to

one.

**[16:25]** And what exactly has happened? You have replaced it with the extreme intense values. This is okay with respect to binary. But what if you have an a gray scale image? In the gray scale, what we consider, here the values will be between zero to 255, right? Now, say for example, you have an image where the values are like this. Can you see? 128 is replaced it with a zero. So, zero here is a black, which means it is a pepper. And 130 is replaced it with a 255 extreme intensity. So, 255 is white. So, now some values are changed to zero and some values are changed to 255. With this, can you able to imagine if you have an image like this, there will be so many white and a black dots.

**[17:19]** So, will it not be having white and black dots like this? So, when it is white and the black dots, is it not a noisy noisy image or noisy data, right? So, this is what is salt and pepper. So, what we are saying, you know, the whole intuition here is the original input is corrupted. The corrupted input is passed to an encoder or autoencoder. And the original from the output from the decoder side we get the reconstructed one. That reconstructed is compared to the original clean one, but not the corrupted one. With this, we are enforcing the model not to relax because the task is heavy. Why the task is heavy? Not the The model should not compare it with the corrupted input. The model should compare it with the original clean one.

**[18:13]** So, when it has to compare it with the original clean one, definitely it cannot relax. That when it cannot relax, it is like, you know, a regularization. Right? Okay. Now, with this, there is one important concept. What is that important concept? What is our ultimate aim? Now, look at this. Here it is an corrupted input image. The original was supposed to be a clean three. But, by adding salt and pepper, there are so many, you know, cuts in between. So, there is a lot of information which is lost here. Not only salt and pepper, you take any noise any noise when it is being added, the original information, some part of original information is lost.

**[18:59]** Now, when it is reconstructing, should it construct a three with so much of gaps? No. It has to give us the original clean the only. If it has to get the original clean the three, how do the model knows that here there is data? Here what data should come? What data should come here? How will the model understands that? Can you able to understand this? How will the model learns? Right? So, that is what is the point of discussion here in this particular. And towards the end I'll connect you to that neighboring a pixel dependency. So, you have understood the task here. The task here is you have a corrupted input. Now, our ultimate aim is to get a clean three. If you have to get a clean three, how will the model

**[19:48]** knows that here it should be a curve only? It should be this curve. It should be this curve. How will the model knows it? So, if the model can learn that, reconstruction will be correct three, right? So, now look at this one more image. Here

all these pixel values, all of these are the pixel values, right? Now, you know, whenever you consider an image, image is made up of pixel values. And every pixel is a dependent on its neighboring a pixel. Do you agree to this? Which means, now say for example, if you consider the face a photo here, all the pixel values which are in this particular region will be all similar. And all the pixel values which are here will be similar, right? Because then only you

**[20:38]** can form a particular shape. So, it cannot So, all the neighboring pixels are not independent to each other. They are strongly dependent. There is a high correlation. Can you agree to this? All this are highly correlated. This is highly correlated. All these are correlated. So, what we are trying to say is in an image, the value of a pixel is usually not independent, right? We cannot say they are not independent. They are highly correlated. Correlated with what? With the its neighboring pixels. I hope you agree to this concept. Just imagine an a cat image. In the cat image, when you take the tail, so all the values there will be of a similar one. When you consider the legs, all the legs pixel values will be

**[21:31]** similar. When you take the body, the you know face, the eyes. So all the values, all the neighboring one will be dependent to each other. The neighboring pixels are highly correlated. This what you need to understand from this particular. Suppose, let's consider one small pixel one one pixel value. Let us consider this particular pixel value. And that you know, you can even consider this. So when you take this particular pixel value, now this is where this particular pixel is located.

**[22:10]** This particular pixel is located from that ith you know and this jth one. So the coordinates of this is  $i, j$ , right? So if you consider  $x_{i,j}$ , which means it denotes one pixel value at a location  $i, j$ . And what will be that pixel value? That pixel value, if it is a binary image, that pixel value will be zero and one. Suppose if it is an gray scale or you know gray scale images, the value will be either between zero to 255 or if you have a normalized it, the value will be zero one.

**[22:50]** Zero to one, right? So this is what is one pixel value when you consider an binary or an gray scale. Now, for example, if a pixel if that particular pixel belongs to an curved black you know stroke. Now, let us assume this is the pixel value. Right? If this is the pixel value and if this pixel value belongs to this curved one, then if at all if you have a pixel here or if you have a pixel here, there is a very high possibility that those also belongs to that curved shape only. That is what is being explained here. If if a particular pixel belongs to an curved black stroke, then all its nearby pixels are most likely to belong to a same stroke. Do you agree to this?

**[23:46]** Yes? This is what we are slowly connecting to this neighboring pixel dependency. Now, the question is what if that pixel is missing? Ultimately, our aim is to reconstruct the original clean one. Now, [snorts] because of this noise, some pixel values are missing. Now, how do we get how do we get that data back? How do we

get the data back? Can you think? So, now can I say if one pixel value is missing, we have all its neighboring pixels and we know that all neighboring pixels belongs to a same high possibility belonging to a same. So, with the help of neighboring pixel, can you able to tell that missing pixel value? Look at this.

**[24:29]** Suppose if this is all, you have your  $x_{i,j}$  here. What if this is a missing? If this is a missing, now can I infer that missing one from its a neighboring? If I know the values of all of this, can I able to know what exactly what will be the probability of this particular value? Right? So, denoising autoencoder, it is just not reconstructing the data, it is a learning that you can reconstruct the data by using a neighboring a pixel dependency. So, wherever there is a missing value, from its neighboring one, I can able to understand that missing value. This is what denoising autoencoder learns all about.

**[25:15]** So, if the center pixel value is a corrupted, if this particular value is corrupted or missing, then we can easily infer this from the eight surrounding eight pixel values. 1 2 3 4 5 6 7 8. From this eight, I can able to tell what exactly is this particular value. So, can I ask this? Given the neighboring a pixel, if I know all these neighboring a pixel, what is the probability distribution of that a center pixel value? So, this is what?

**[25:51]** Now, in case, if this concept is still not a clear to you, I'll give you one simple numerical example only for understanding a purpose. However, in a practical application, we are not going to use this method. I'm just showing you only for understanding how that neighboring dependency will help in understanding that missing value. Again, I'm telling, in the real-world application, we will not be using this particular technique. This is only to get an understanding of neighboring pixel dependency.

**[26:22]** Let us consider this particular input image. In this input image, the center value is missing. Now, if the center value is a missing, now we want to fill that missing value. How do you fill that missing value? Can we count one simple direct method? Can we count how many ones are there? How many zeros are there? Number of zeros are two. Number of ones are six, right? So, now, with this, what is the probability that uh uh center value, which is missing belongs to one or whether it belongs to zero.

**[26:56]** How is that you will do? A simple uh straightforward neighborhood uh estimation is uh what is the probability that uh uh the center value is one? So, it is uh since the number of ones are six is six divided by total. I'm getting 0.75. Next, what is the probability that it will be zero? So, zero is a two 2 / 8 is this. Here, the highest value is 0.75, which means that the center value should be or the center pixel value which is missing is one. Right? So, therefore, this is uh the missing value. One uh so, you're going to replace it. Now, you can ask a question, "Ma'am, you're uh at the center you're missing the value. What if you have an image in this uh form?" What if you have an image uh where here

**[27:50]** at the corner uh you have or not at the corner, you have not at the center in

some other position you have a missing value. Can you able to take this neighboring uh eight? So, you you will have again this if you consider it as uh center uh uh so, can you consider for this eight uh neighboring uh pixels? Can we consider uh eight neighboring uh pixels uh for this? So, model will consider these values. And then, it will find out where there is a neighboring one. So, uh this is how uh in the real world we will not be using, but just to understand the neighborhood, I'm explaining you this.

**[28:39]** So, let's take one more example where uh let us see for this what will be the uh possibility. So, what is that you will do? Just count how many number of zeros are there and ones. So, number of zeros are six and one here are two. So, once again do the same calculation. Now, here for zero the value is high. Therefore, that missing value » [snorts] » is uh filled it as zero. So, this is what uh is a neighborhood. Now, you can tell this is simple binary. What if you have something in the form of a gray scale? So, this is a gray scale image where you don't have in terms of zeros and ones. You have values like this. In that in this case, we cannot count the zeros and ones. What is your approach?

**[29:23]** Uh what will be the missing value here? What will be the approach? Right? Straightforward, you will take the average. So, what you will do? You will first list all the neighboring pixels. So, the neighboring pixels are 120, 125, 130, then 123, 127, and so on and so forth. Then, we are going to take the mean of it. So, take the add up all of this and then take the average. So, the average here is 124.625. If there are any calculation errors, you can correct it. So, since there are since the value is 124.6, can I replace this with 125? So, after rounding off, I'm getting that missing value as 125. And therefore, you see the filled patch will be 125.

**[30:14]** Now, can you able to see this? This 125 is a closely matching to all its neighboring one. Uh So, this is what is the dependency. As I said, in the gray scale neighborhood usually have a same intensity values. So, you can missing value can be estimated from the nearby intensities. So, uh all the neighbors are in between 120 to 130, the center value is also likely to be around that particular range only. So, this is whatever the two examples in this is slide and in the previous slide we have seen, it is just only for the understanding uh purpose, but not uh the practical application. In the practical application, how is that you are doing it? Can you recall it from our week one CNN?

**[31:08]** Uh uh so, what is uh that you will be doing it? Um there is an image. That image is a pass through to an CNN because it's an image we are considering as an input. When you pass it through a CNN, CNN has to extract the features. What features it will extract? How it will extract? It will extract by performing a convolution operation with the help of a kernels. So, in the real world, whenever you're using a denoising and then if you're using an image as an input, we will not be using whatever the two examples we have seen in this slide and in the previous slide. We will be using

an convolution filter for for extracting the features and for learning the neighborhood.

**[31:51]** So, let us see this how exactly. Now, this is what is the pixel value. And in the practical implementation, how do we implement it? Of course, starting will be adding the noise. And when it has to extract the features, the features will be extracted using this particular line. Is it not similar to what we have seen in the uh your week one where it is a CNN? We have a 32 such a filters. Each filter is of size of 3 cross 3. We have a ReLU activation. You have applied padding for that you are passing an input. So, exactly same as our week one CNN concept. I'm showing you here once again. Let us assume there is an input that we have considered. This is the kernel that we have taken. What is the

**[32:35]** first step? Is it not the kernel is applied onto this input? Now, let us assume this is the missing value. The zero here is the missing value. How is that you will be doing? Is it not a simple calculation? You will multiply three with one, five with the zero, element wise multiplication, and then you will add up. So, convolution is element wise multiplication, and then you will sum up all of this. You're getting 17.

**[33:05]** So, even though there is a missing value here, and next when you stride at, is it not you get this value? When you get this value, here is a missing value. Even when there is a missing value here, is it not you're considering all of its neighboring eight pixels? And then you're able to tell, if this is the case, what could be this possible value, right? Then, you will stride. When you come down, is it not you'll consider this?

**[33:34]** You have a missing value here. So, when you have a missing value here, you're considering all its neighboring values. Then you will consider the next. So, wherever there may be a missing value, you will consider its associated neighboring eight pixels, and then you will be not eight here in this particular example, because the window size is a three cross a three, I'm telling you neighboring eight. But otherwise, based on how the parameters have been defined, your hyper parameters, accordingly it will consider its neighbor.

**[34:07]** Right? So, did you get this concept overall? What exactly we have seen? The input is corrupted. So, in the process of corruption, when you reconstruct it, there is a high possibility that so many pixel values are missing. So, when it has to reconstruct, that missing values also should be filled. Then only you get the correct clean three image. So, when the values are missing, how is that it is going to fill it? It is going to use the concept called the neighborhood dependency. How is that neighborhood dependency is what we have seen in this particular example.

**[34:45]** Right? So, with this, let's formulate what is denoising. What is the first step? The first step is you have to corrupt the input. How is that you will you will take the original clean image and you will corrupt it by adding either the Gaussian noise or you'll be adding the masking or you'll go for salt and pepper or you can use some

other technique, but intentionally corrupt the input. Now, that corrupted input is a passed to an encoder. What is the work of an encoder? Same as now rest of it is same as your the original whatever we have discussed.

**[35:21]** That input you see what input we are passing. This corrupted input gets multiplied with the weight and then the bias and then you're passing it to an activation function and you get the hidden representation. If you have one hidden representation, directly you get one. If you have multiple hidden layers, then the output of last hidden layer will be the latent representation  $H$ . Now, that is a passed to an decoder. What is a decoder?

**[35:49]** Decoder will take a  $H$  that gets multiplied with the weight bias and then you're passing it to an activation function. If it is intermediate hidden layers at a decoder, you'll have a radio activation or non-linear. If it is a last one, then you see based on what kind of input, accordingly place an activation function. So, therefore, this is an output activation function. Okay? What is that we are getting? We are getting  $\hat{X}$ . [laughter]  $\hat{X}$  is the output image. But, what is our ultimate goal? Goal is to reconstruct well. If it has to reconstruct well, the error has to reduce. If error has to reduce, what is the original ground truth? The original ground truth is the original clean image. Now, you can imagine the original

**[36:38]** clean image is this. And what you have passed is this. Now, when you are reconstructing, the reconstruction will be something like this initially. Now, you are comparing it with this original ground truth. There is a huge difference, then it is going to optimize the weights properly. Then, for the next iteration, probably you would have got this. Again, there is an loss. You're comparing it the original. Again, the loss is too much. You have to optimize the weights. Over a period of time, when your ground truth is this and what you have constructed is this. Over a period of time, in the training, you will end up getting an um you know, clean image. If not exactly similar to that, more nearer to that. Then, during

**[37:37]** the testing, even when you pass the corrupted image, what model is going to give us is an clean image. So, this is all about denoising. And so, decoder part we have seen. And the next is the full pipeline is you have an input which is corrupted and it is reaching a to tilt. That is passed to an encoder. You are getting an  $H$ . That is passed to an decoder. You are getting an reconstructed one. The model learns this. Can you look at this?

**[38:08]** This is what is the input corrupted one when it is a passed to this. And ultimately, the comparison is between these two. Reconstructed and the original clean image. Now, and what is the primary training objective? You have to minimize the loss between the clean input and then the reconstructed one. And that loss can be either MSE if you are using a continuous valued data or it can be binary cross entropy if it is binary data or a normalized data. And what is the hidden objective

here is the neighborhood connection.

**[38:50]** How is that you get the neighborhood connection? If at all if there are some pixel values, the reconstructed pixel value at a location  $I \{ \text{comma} \} J$  is obtained of its local neighborhood. This. So, we have seen this with one example in the previous slides. So, this is one of the most powerful model which is a denoising autoencoder. The application of this is wherever you have an corrupted data. When you have a corrupted data, it will help you in getting a clean data. So, for that take the data intentionally corrupted so that the model will learn properly.

**[39:31]** Now, the last topic here is we have called this as a regularized variant. Now, what exactly is a regularized one? How a denoising autoencoder has become regularized one? So, in a standard autoencoders, what exactly has happened? The clean input is passed to you know model and we have obtained  $H$  and from there we have passed to decoder and we got  $\hat{X}$ . And our ultimate aim is reconstructed should match to this. Right? But in denoising autoencoder, what is happening? Original clean image is being corrupted. And how you have corrupted? Intentionally you have corrupted. It is not you are taking a corrupted data. Whatever the original clean data, that clean data you will be adding the noise. You'll see that in

**[40:21]** hands-on where the noise will be added to any images. So, it becomes a noisy data. That noisy data is passed to a model and you're getting a reconstructed one. But, aim Look at this. Here you were comparing it with  $X$ . Here also uh you are comparing it with the  $X$ , but look at this. This  $X$  is not equivalent to the corrupted one. Are you getting this? So, now if at all even if you have an you know over complete architecture, let us assume you have an over complete architecture like this.

**[41:01]** Now, what is over complete? The What is the problem? Maybe there is an identical you know identity mapping here. So, no proper learning is happening. In case, if you do this way in this particular model, if you allow it to happen, input what you have passed is a noisy input. Then, if you if it goes as it is, if you copy as it is, then output is also a noisy one. Then, this noisy you're comparing it with the original clean one. You will never minimize the loss in that case, right? So, where is the question of model relaxing here? The model cannot relax. There is a lot of restriction here where the corrupted one should be compared it with the original clean one. So, this is what is you know why it provides regularization is

**[41:50]** because of the corruption, it removes the shortcut of direct copying. And then, if the model must infer all the missing values from the neighboring one or from the context. So, this encourages the model to learn the features rather than directly copying it. So, this is what is a denoising autoencoder, One of the important variant of auto encoders. In the next session, we will see the second variant which is a called an sparse auto encoders. So with this we have come to an end of this particular session and in the next session we will see this sparse. Thank you.